CPSI 28003 - 01 and 9S1 – Algorithms

Take-Home Final Exam (Open Books and Notes)

10:30 AM – 12:30 PM, 12/11/2025, Thursday

Please submit your answers to the blackboard

Instructor: Dr. Chia-Chu Chiang

Department of Computer Science
University of Arkansas at Little Rock
2801 South University Ave.
Little Rock, Arkansas 72204-1099

Total Points: 100 Points
Total Pages: 13

Name: Isaac Fredricks T#: 4922 (Last 4 digits)

Part I: Multiple Choice Questions (10 Points)

1. b _____ (2 Points) In hash table, linked lists are used in
   (a) Linear probing
   (b) Separate chaining
   (c) Rehashing
   (d) All of the above
   [ANSWER]

2. b _____(2 Points) Which of the following postfix expressions is **valid**?
   (a) 3 6 2 + 7 5 + *
   (b) 4 3 + 7 * 5 + 8 +
   (c) 3 5 + 2 + + *
   (d) 5 + 9 2 5 + *
   [ANSWER]

3. (2 Points) Parentheses are eventually are not required in the postfix expression in the conversion of infix to postfix expression.
   (a) True
   (b) False
   [ANSWER]

4. (2 Points) When parentheses are in the conversion of infix to postfix expression like the one below,

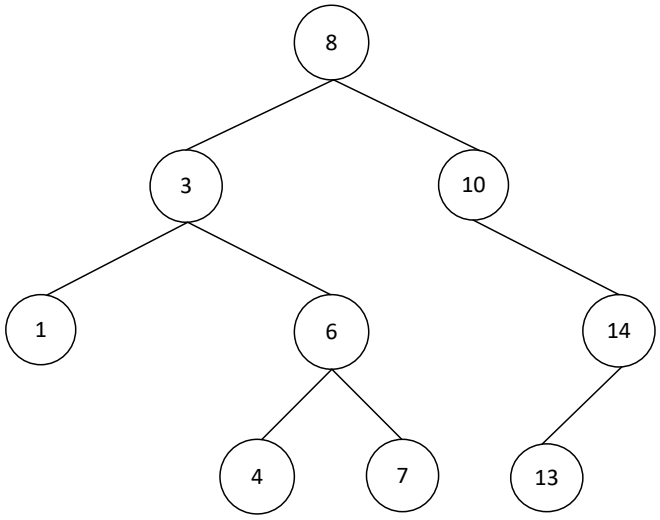
   (( ) (( )) ( ))

   What is the maximum number of parentheses that appear on the stack at any instant?
   (a) 2
   (b) 3
   (c) 4
   (d) None of the above

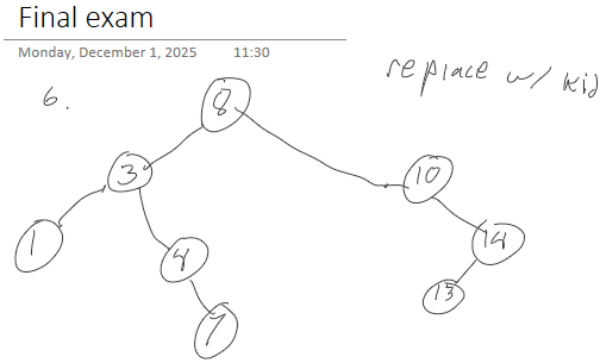
5. (2 Points) Which of the following statement about a binary search tree is **NOT** true?
   (a) Every node in the binary search tree has at most two children.
   (b) Every node in the binary search tree has exactly one parent.
   (c) The binary search tree has no cycles in it.
   (d) A balanced binary search tree is a binary tree where the left and right sub-trees of every node differ in height by no more than 1.

Part II: Short Answers (50 Points)

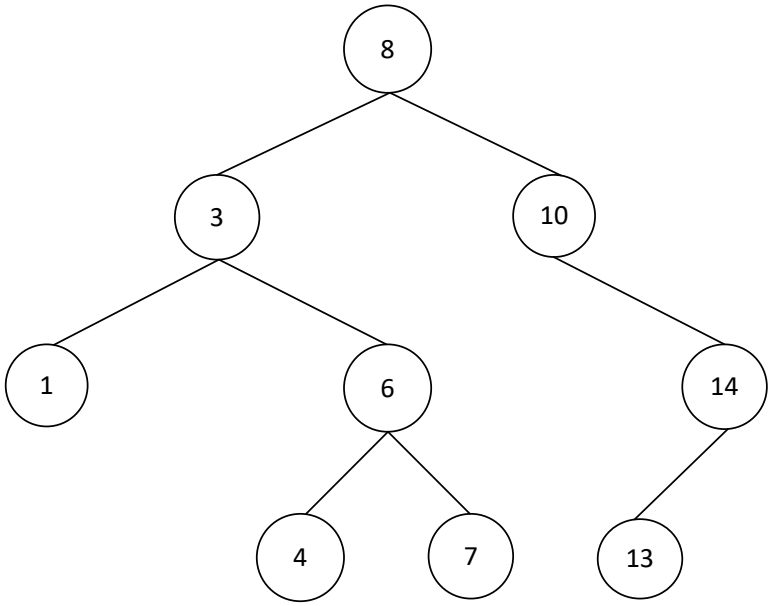
6. (5 Points) Given the following **Binary Search Tree**,



Draw the **Binary Search tree** after deleting the value 6. Use one of the methods discussed in the class for deleting the node.
[ANSWER]



7. (5 Points) Given the following **Binary Search Tree**,



7a. (2 Points) What are the **<u>internal nodes</u>** of the above binary search tree?
[ANSWER] 8 3 10 6 14

7b. (3 Points) Write the order of the nodes visited in **<u>in-order traversal</u>**?
[ANSWER] 1 3 4 6 7 8 10 13 14

8. (5 Points) In a **Binary Tree**, a node may have at most 2 children. In the following questions about binary trees, the height of a tree is the length of the longest path. A tree consisting of just one node has height 1.

8a. (3 Points) What is the <u>maximum</u> number of nodes in [illegible] height 3?
[ANSWER] 7

8a) $h=3,\ |V|=2$
$2\cdot|V|+1 \le n \le 2^{|V|+1}-1$
$2\cdot 2+1 \le n \le 2^{2+1}-1$
$5 \le n \le 7$

8b. (2 Points) What is the <u>minimum</u> number of nodes in [illegible] height 3?
[ANSWER] 5

9. (5 Points) In a **Full Binary Tree**,

9a. (3 Points) What is the <u>maximum</u> height of a full binary tree containing 7 nodes?
[ANSWER] 3

9)
$\log_2(n+1)-1 \le |V| \le \frac{(n-1)}{2}$
$\log_2(7+1) \le |V| \le \frac{7-1}{2}$
$3 \le |V| \le 3$

9b. (2 Points) What is the <u>minimum</u> height of a fu[illegible] 7 nodes?
[ANSWER] 3

10. (10 Points) Draw the final result after inserting keys **24, 25, 50, 38, 12, 90** into a hash table with collisions resolved by (a) linear probing, (b) chaining. Let the table have 13 slots with addresses starting at 0, and let the hash function be h(k) = k mod 13.

10a. (5 Points) Linear Probing

0	1	2	3	4	5	6	7	8	9	10	11	12
50	38	12	90								24	25

[ANSWER]

10b. (5 Points) Separate Chaining
[ANSWER]

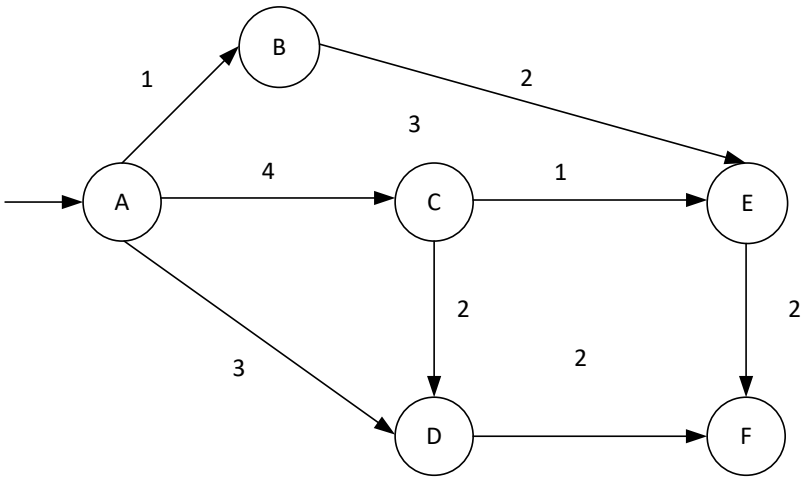
11= 24->50
12=25->38->12->90

11.(10 Points) What is the order of growth of the worst-case running time of each of operation below? Write down the best answer in the space provided, using one of the following possibilities.

O(1) O(log N) O(N) O(N Log N) $O(N^2)$

(a) O)N Log N) insert an item to a binary search tree
(b) _____ delete an item from a binary search tree
(c) O(1) push an item onto a stack.
(d) O(N Log N) time complexity of Merge sort as the sequence of elements partitioned equally in halves.
(e) O(n^2) time complexity of QuickSort as the largest element of the sequence selected as the pivot.

12.(10 Points) Consider the following directed, weighted graph,



Step through Dijkstra's shortest path algorithm to calculate the single-source shortest paths from A to every other vertex. Show your steps in the table below. **Cross out old values and write in new ones, from left to right within each cell**, as the algorithm proceeds. Also list the lowest-cost path from vertex A to vertices including vertex F. **Fill in the blanks**. [ANSWER]

Vertex	Shortest Distance from A	Previous Vertex
A	0 –	–
B	A->B=1	
C	A-C=4	
D	A->D=3	~~A->C->D=6~~
E	A->B->E=3	~~A->C->E=5~~
F	A->D->F=5	~~A->C->E->F=7,~~ ~~A->C->D->F= 8,~~ ~~A->B->E->F=5~~

13.(5 Points) Given the **Node.h** and **BinarySearchTree.h** files below,

```
class Node
{
  friend class BinarySearchTree;
public:
  Node();
  Node(int val);
  ~Node();
private:
  Node* left;
  int value;
  Node* right;
};
class BinarySearchTree
{
public:
  BinarySearchTree();
```

```
    ~BinarySearchTree();
    // insert a new node to the binary search tree
    void insert(Node* node);
    void inOrderTraversal(Node* root);
    Node* f();
    int g(Node* root);
    int h(Node* root);
private:
    Node* root;
};
```

13a. (5 Points) What does the following code f do on a **Binary Search Tree**? **DO NOT JUST EXPLAIN THE CODE.**

```
Node* BinarySearchTree::f() const
{
    Node* currentNode;
    currentNode = root;
    if (currentNode != NULL) {
      while (currentNode->left != NULL)
          currentNode = currentNode->left;
    };
    return (currentNode);
}
```

[ANSWER]

It finds the leftmost (smallest) node

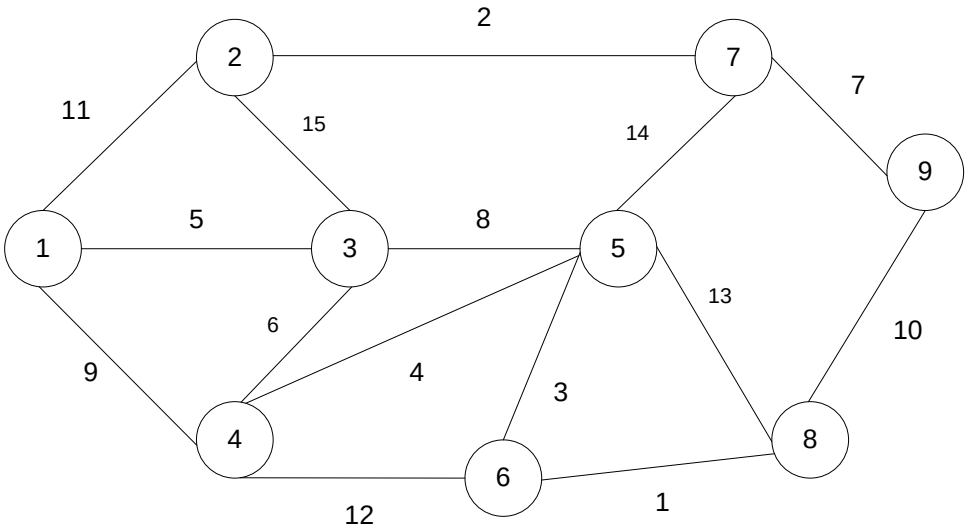
14. (5 Points) Suppose that we have a **binary search tree** and we insert a node with the value x into the binary search tree that does not already contain the value of x. If we then immediate delete x from the tree after the insertion, will the tree be identical to the original one? **Please justify your answer to obtain the credits.** If you answer no, please give a counter example. Draw the tree help the justification if necessary. [ANSWER]

Yes, the binary search tree once removed of the node value x, it will be identical to the original one. This is because we have not attached any child nodes to it, so we do not have to replace the removed nodes with one of its child nodes.

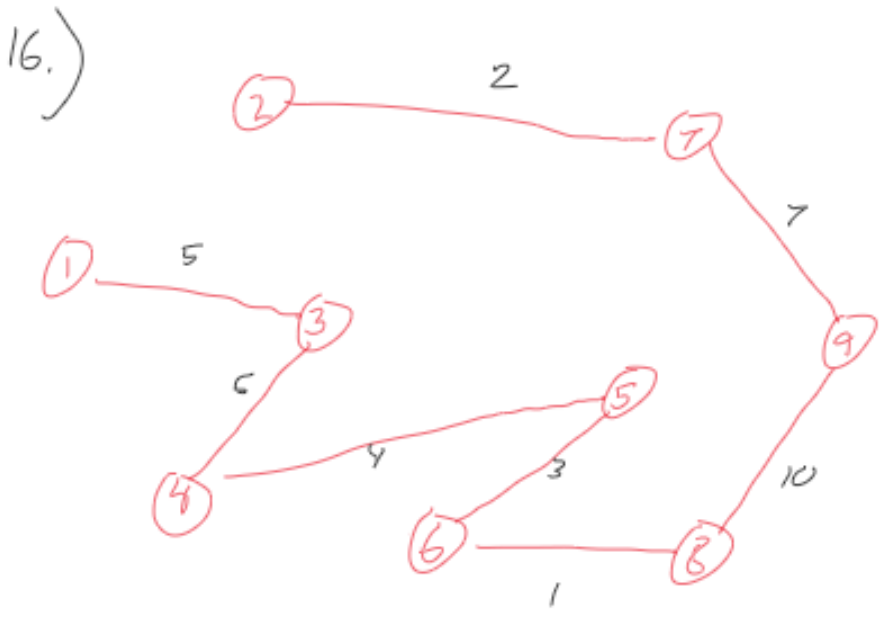
15. (5 Points) Given a sequence of integers for quicksort, when the worst case of time complexity $O(n^2)$ occurs? Explain why this makes Quicksort perform so badly.
[ANSWER]

The quicksort performs the worst when the pivot is either the smallest or largest element. It performs well on the side with no elements, but performs significantly worse when starting from the smallest or largest element bcause the sort has to search linearly for items smaller or larger than the pivot. Because it starts at the smallest or largest element, it goes from T(0), T(1),...,T(N) or T(N), T(N-1),...,T(1), T(0) respectively.

16. (5 Points) Given the following graph, use **Prim's algorithm** to compute the Minimum Spanning Tree (MST) of the graph. Write down the edges of the MST in sequence based on the Prim's algorithm that adds them to the MST. An edge is denoted in the format (node1, node2).



[ANSWER]



Part III: Programming Code (20 Points)

17. (10 Points) Given the **Node.h** and **BinaryTree.h** files below,

```
class Node {
   friend class BinaryTree;
public:
   Node();
   Node(int s);
   ~Node();
   int getElement();
private:
   int element;
   Node *left;
   Node *right;
};


class BinaryTree {
public:
   BinaryTree();
   ~BinaryTree();
   void insert(int x);
   void printTree();
   void printTreePreOrder();
   void printTreeInOrder();
   void printTreePostOrder();
   void remove(int x);
   bool isBinarySearchTree();
private:
   Node *root;
   void showTreePreOrder(Node *p);
   void showTreeInOrder(Node *p);
   void showTreePostOrder(Node *p);
   int findMinimumValue(Node *p);
   int findMaximumValue(Node *p);
   bool checkBinarySearchTree(Node *p);
};
```

Given two helper functions **int BinaryTree::findMinimumValue(Node *p)** and **int BinaryTree::findMaximumValue(Node *p)** functions that return the smallest value and the largest value from a binary tree rooted by p. Use these two helper functions to implement the following function **bool BinaryTree::checkBinarySearchTree(Node* p)** which returns TRUE if the binary tree is a binary search tree; otherwise FALSE. The **bool checkBinarySearchTree(Node *p)** is invoked by **isBinarySearchTree()** which is shown below.

```
bool BinaryTree::checkBinarySearchTree(Node* p){
// FILL IN THE BLANK (2 Points per Blank)
if (p == NULL)
    return(true);

if (p->left!=NULL && findMaximumValue(p->left ) > p->element)
    return(false);

// false if the min of the right is <= than us
if (p->right!=NULL && findMinimumValue(p-> right ) <= p->element)
    return(false);

// false if, recursively, the left or right is not a BST
if (!checkBinarySearchTree(p->left )
|| !checkBinarySearchTree(p->right ))
    return(false);

// passing all that, it's a BST
return(true );
}
```

18. (10 Points) Given the **Graph.h** and **Graph.h** files below, a graph could be represented as a collection of nodes, each of which contains a list of adjacent nodes that it is directly connected to by a path (see the example below). To find paths or otherwise explore the graph it is also helpful if each node contains a Boolean value indicating whether the node has been visited while processing the graph.

```
// Node.h
class Node {
  friend class singlyLinkedList;
  friend class Graph;
public:
  Node();
  Node(int val);
  ~Node();
  int displayValue();
private:
  int value; // label vertex number
  bool visited; // node has been visited or not
  Node *next;
};
```

```
// Graph.h
class Graph {
  friend class singlyLinkedList;
public:
  Graph();
  Graph(int value);
  ~Graph();
private:
  Node *pGraph;
};
```

Assume we have 16 vertices (nodes) named from 0 to 15 and an adjacency list structure shown below is used to represent the graph with the nodes.

pGraph[0]→value = 0

pGraph[1]→value = 1

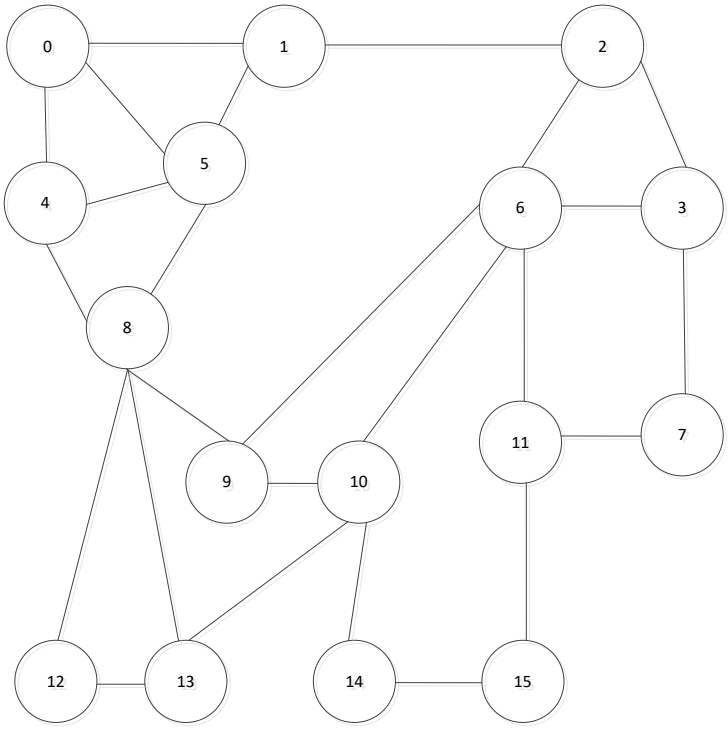
...

Vertex No	Edge List
0	4 → 5 → 1
1	0 → 5 → 2
2	1 → 6 → 3
3	2 → 6 → 7
4	8 → 5 → 0
5	1 → 0 → 4 → 8
6	2 → 3 → 11 → 10 → 9
7	3 → 11
8	12 → 13 → 9 → 5 → 4
9	8 → 10 → 6
10	14 → 13 → 9 → 6
11	7 → 6 → 15
12	13 → 8
13	10 → 8 → 12
14	15 → 10
15	11 → 14

tempNode = pGraph[0].next which is tempNode 4
tempNode = tempNode→vext which is 5
tempNode = tempNode®next which is 1
tempNode = tempNode®next which is NULL

tempNode = pGraph[1].next which is tempNode 0
tempNode = tempNode→vext which is 5
tempNode = tempNode®next which is 2
tempNode = tempNode®next which is NULL

The graph represented using the above structure is shown below.



Complete the following code of method countNodes below so it returns the number of nodes that can be reached directly or indirectly by some path from the starting node. The count should include the starting node. **You should assume that at the beginning of the search the visited fields in all nodes are set to false.** Please fill in the blanks.

```
// Return the number of nodes recursively that can be
// directly or indirectly reached from node n. The original
// starting node should also be counted once. */


int Graph::countNodes(int vertexNumber)
{
 int tempCount =0_____; // init tempCount for # of nodes
                          //(2 Points)

 if (pGraph[vertexNumber].visited == false_____)(2 Points)
     return 0;

 pGraph[vertexNumber].visited = true;
 tempCount = tempCount+1_____(2 Points);

 // loop through the edge list of the vertex
 Node* tempNode = pGraph[vertexNumber].head_____(2 Points);
 while (tempNode != NULL)
 {
  if (pGraph[tempNode->value].visited != true)
   tempCount = tempCount + 1_____(2 Points);
  tempNode = tempNode->next;
 } // while

 return tempCount;
}
```